TECHNICAL REPORT TR-01

Hop3: A Sovereignty-First Platform-as-a-Service for Single-Server Deployment

Abilian SAS · April 2026 · 2.0 (INTERIM)

Contact hop3@abilian.com
Project Nix Integration for Hop3, NGI0 Commons Fund
Status Interim technical report. A final version is scheduled for late 2026 following completion of the quantitative evaluation phase described in §5 and §7.

Contents

> **Superseded material (noted 2026-07-22).** This report describes the system as of April 2026 and is left as that record. One part of it has since been overtaken: the reproducibility taxonomy in §3.5 defines Tier-2 as a `__noChroot` escape that grants the build network access, and §6.3 defers dependency vendoring as too costly to be worth it. Vendoring was subsequently implemented for all six ecosystems, so `__noChroot` no longer appears in any template, every template builds offline in the sandbox against a lockfile-pinned dependency set, and the tiers now classify provenance rather than hermeticity. Read §3.5 and §6.3 as a record of where the work stood at that time. The current taxonomy is ADR 058 the measured results are in the final report.

Abstract

We present **Hop3**, an open-source Platform-as-a-Service (PaaS) designed for deploying and managing web applications on a single server. Hop3 occupies a position between two dominant points on the deployment spectrum: heavyweight container orchestrators (Kubernetes and derivatives), which impose multi-node infrastructure and substantial operational overhead, and ad-hoc provisioning scripts, which lack reproducibility and lifecycle support. The system provides a git-push-based deployment flow and a CLI/TUI interface backed by a JSON-RPC server; internally, a plugin architecture separates builders, language toolchains, deployers, addons, and reverse proxies. A central contribution of this report is Hop3's **Nix-based reproducible build path**, an alternative to Docker-first deployment that exercises three tiers of reproducibility (nixpkgs-source, `__noChroot`, and prebuilt-artefact) through eight code-generation templates distilled from observed packaging patterns. We describe the architecture, the plugin decomposition and its rationale, the configuration model, the Nix integration and its runtime contract, the backing-service abstractions (PostgreSQL, MySQL, Redis, S3-compatible object storage), and the diagnostic infrastructure that captures distributed failure context before cleanup. We report a preliminary qualitative evaluation covering 99 end-to-end application-variant deployments across 20+ real open-source applications and four build strategies. Quantitative benchmarking against K3s, Docker Compose and bare uWSGI is deferred to the final report.

Keywords: Platform-as-a-Service, single-server deployment, Nix, reproducible builds, digital sovereignty, self-hosting, deployment automation, plugin architecture.

1. Introduction

1.1 Motivation

Deploying a web application onto a server today typically requires a choice between two unsatisfactory options. On one end, container orchestrators such as Kubernetes [1] and its single-node derivatives (K3s, Minikube) provide reproducible, declarative deployments but at the cost of multi-component control planes (etcd, kubelet, scheduler, controller manager), YAML manifests, and operational complexity that is disproportionate to the common case of running a small number of web applications on one host. On the other end, ad-hoc provisioning scripts and hand-written `systemd` units offer simplicity but lack reproducibility, lifecycle support, and any abstraction over backing services.

A large class of real operators (small and medium businesses, managed service providers, non-profits, educational institutions, and independent developers) sits between these two extremes. For this class, the dominant deployment pattern is still a single machine (or a small number of machines) running multiple applications side by side, often with a reverse proxy in front and one or more shared database instances behind. The operator wants automated deployment, basic lifecycle management, and reproducibility, without paying the cost of a distributed control plane.

Hop3 targets this gap. It provides a PaaS with a git-push deployment flow comparable to Heroku and Dokku [2,3], a plugin-based extensibility model, authentication on every control-plane request, and, as the principal technical contribution of the current project phase, a Nix-based alternative build path that offers a credible reproducible-build story without requiring a container runtime at all.

1.2 Contributions

This report documents the state of the Hop3 project as of April 2026 and describes the following concrete artefacts:

1. **A plugin-based PaaS architecture** with seven orthogonal plugin categories (builders, language toolchains, deployers, addons, proxies, operating-system implementations, health checks) that together cover the full deployment pipeline. The decomposition is motivated in §3.3. The system is implemented in Python on top of Litestar (ASGI), Pluggy, SQLAlchemy, and Dishka dependency injection.
2. **A Nix integration subsystem** consisting of (a) a hand-crafted `hop3.nix` build path, (b) an eight-template auto-generation system covering the recurring packaging patterns observed across the application catalogue (see Appendix B), and (c) a runtime contract (`$out/hop3/runtime.json`) that decouples the Nix derivation from the Hop3 runtime and allows the two to evolve independently.
3. **A reproducibility taxonomy** distinguishing three tiers of Nix-based build: Tier-1 (nixpkgs-source, fully sandboxed), Tier-2 (`__noChroot`-escape for tools needing network or specific devices), and Tier-3 (prebuilt artefact wrapped for runtime use). The taxonomy is surfaced as explicit per-template metadata and lets operators and auditors reason about the reproducibility guarantees of a given deployed application.
4. **A uniform backing-service abstraction** covering relational (PostgreSQL, MySQL/MariaDB), key-value (Redis), and object-storage (S3-compatible) services. Each addon is responsible for its own provisioning, credential injection, teardown, and connectivity from both host-process and containerised consumers. The S3 addon is built on a pluggable backend interface so that the underlying implementation (MinIO, Garage, etc.) can be swapped without application changes.
5. **A structured-diagnostics pipeline** built around a `Diagnosis` record (`component, action, reason, hint`) that replaces bare tracebacks at every major failure point in the deployment pipeline. On end-to-end test failures, per-test runtime diagnostics are captured before cleanup destroys the application state, yielding self-contained failure reports.
6. **An end-to-end test corpus** of 99 real-application variants (Appendix C) spanning PHP, Node.js, Go, Python, Java, Ruby and Rust across four build strategies (Docker, native uWSGI, hand-crafted Nix, template-generated Nix), running on a remote VPS.

The claims of this report are primarily qualitative and architectural. A quantitative evaluation (§5.4, §7) covering control-plane footprint against K3s, Docker Compose and bare uWSGI, deployment latency, and Nix closure versus Docker image size is planned for the final report later in 2026.

1.3 Report Structure

§2 situates Hop3 with respect to related work (container orchestrators, existing PaaS systems, Nix-based deployment). §3 describes the overall system design, covering architecture, deployment lifecycle, the plugin decomposition, the configuration model, the Nix integration, and the security model. §4 covers implementation, organised by subsystem. §5 reports the preliminary qualitative evaluation and lists the measurements that remain outstanding. §6 discusses limitations, design trade-offs, and threats to validity. §7 concludes and lays out future work. Appendices collect the glossary, Nix template reference, application catalogue, and NGI deliverable mapping.

2. Background and Related Work

2.1 The PaaS Spectrum

Container orchestrators (Kubernetes [1], Nomad, Mesos) are designed for multi-node clusters running workloads that require service discovery, scheduling, self-healing, and rolling upgrades across failure domains. They provide strong guarantees for this case, at the cost of a control plane that is itself a distributed system. Single-node variants (K3s, Minikube, Docker Swarm in single-manager mode) reduce the control-plane weight but inherit most of the operational surface: containers, pods, kubelets, service accounts, RBAC, admission controllers.

At the other end, single-server PaaS systems predate the container era: Heroku's "git-push deploy" model [4] inspired a family of open-source re-implementations, most prominently Dokku [2] and Piku [3]. These systems deploy via buildpacks or simple build-and-run scripts, target a single host, and are operationally much lighter than Kubernetes. Their limitations are (a) a loose reproducibility story (buildpacks resolve dependencies at build time against the network), (b) limited first-class support for backing services beyond what a cooperating operator wires up manually, and (c) ecosystem fragmentation (competing buildpack registries with divergent compatibility matrices).

CapRover [5] sits between these poles: it deploys via Docker, provides a web UI, and offers managed backing services, but inherits the Docker attack surface and image-size footprint, and it binds its deployment story tightly to the Docker daemon.

Hop3 occupies the same operational slot as Dokku and Piku (single host, git-push, plugin-extensible) but departs from them in three ways: (i) it offers a Nix-based reproducible build alternative to buildpacks, (ii) it uses a uniform plugin architecture across builders, toolchains, addons, and proxies rather than a category-specific plugin model, and (iii) it treats failure diagnostics as a first-class design concern (§3, §4.6). It does not target Kubernetes-style multi-node orchestration.

2.2 Reproducibility: Containers Versus Nix

A deployment artefact is reproducible to the degree that rebuilding it from the same stated inputs produces the same bytes. Container images, as produced by `docker build` or equivalent, can satisfy this property only if every layer in the image is itself content-addressed and every external input (apt repositories, package indexes, upstream registries) is pinned to a digest. In practice, common `Dockerfile` authoring violates both preconditions: `apt-get install` fetches whatever is currently in the configured mirror, and `pip install` / `npm install` resolve against the live state of PyPI or npm. The resulting image is reproducible in *content-addressable form* (the same SHA-256 identifies the same bytes) but not *reproducibly buildable*: rebuilding next week from the same `Dockerfile` is very likely to produce a different image.

Nix [6,7] addresses this by making the build environment itself content-addressed. Every input (source code, compiler, system libraries) is identified by a cryptographic hash of its own inputs, recursively. Builds run in a sandbox with no network access except to explicitly declared, hash-pinned fetchers. The resulting derivation is, in principle, reproducible given only the pinned nixpkgs revision. The practical caveats around strict bit-for-bit reproducibility (timestamps, parallel build non-determinism, compiler non-determinism, locale effects) are catalogued by the Reproducible Builds project [8]; they are real but well-characterised.

Nix's guarantee is a *property of the build*. A Nix-derived binary and a Dockerfile-built binary may behave identically at runtime. The guarantee matters for audit, upgrade, and supply-chain attestation. It is irrelevant to operational concerns. The guarantee has a cost: applications whose upstream build system assumes unrestricted network access. Most PHP, Python, Node, Ruby packaging, whose package managers

resolve dependencies during the build, cannot be built inside Nix's strict sandbox without accommodation. Tier-2 (§3.4) exists for exactly this case: `__noChroot`-derivations sacrifice some of the guarantee in exchange for the ability to package a much wider application ecosystem. The guarantee is a gradient rather than a boolean. Hop3′s three-tier taxonomy makes that gradient explicit without papering over it with marketing.

2.3 Prior PaaS Systems

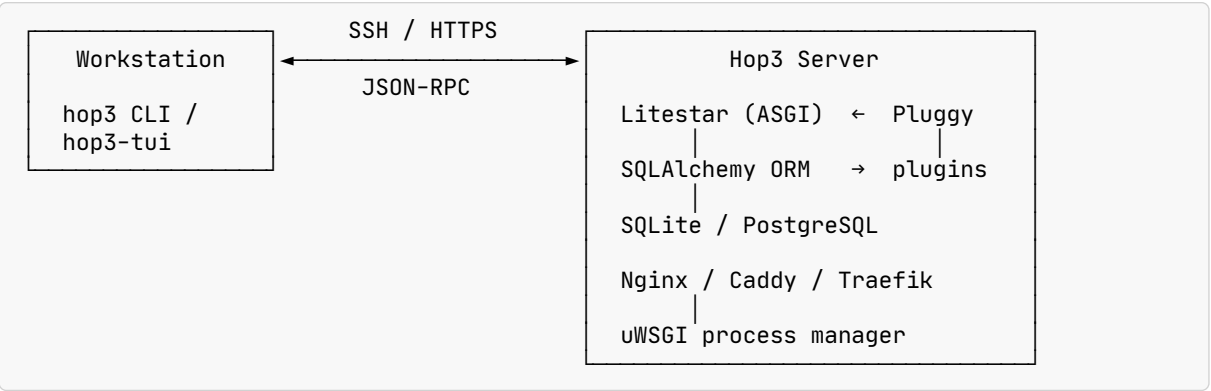
The closest systems in operational model are Dokku [2] (Herokuish-based, plugin system, Docker-backed) and Piku [3] (pure Python, no Docker, uWSGI-backed, minimal plugin model). Hop3 is closer to Piku in internal model (uWSGI-managed processes with per-app isolation) but extends the plugin architecture significantly and introduces Nix as a first-class alternative to the uWSGI-only runtime. Cloud Foundry [9] and its derivatives occupy a larger footprint than Hop3 targets; they are mentioned here as the original source of the "buildpack" concept that Dokku and Piku inherit.

Unikernel-based systems (MirageOS, Unikraft [10]) and WebAssembly-based systems (Wasmtime, WasmEdge, WASI) offer alternative reproducible-deployment routes by replacing the operating system below the application. They have stronger isolation properties and narrower ecosystem coverage. Hop3 can be seen as a "fourth path" alongside containers, unikernels, and WASI: retaining a conventional POSIX runtime (and with it, ecosystem breadth) while gaining content-addressed builds via Nix.

3. System Design

3.1 Architecture Overview

Hop3 is a client-server system. The server runs on the deployment target (typically a VPS or a dedicated machine) and exposes a JSON-RPC endpoint over HTTP(S) or over an SSH tunnel. Clients (the `hop3` CLI and the Textual-based `hop3-tui`) parse user commands locally and dispatch RPC calls to the server. All RPC endpoints require authentication; the CLI acquires a JWT token via an explicit login step (password or magic link) and reuses it for subsequent calls.



Internal components:

- **Litestar** [11] provides the ASGI web framework and exposes the JSON-RPC endpoint.
- **Pluggy** [12] is the plugin host; all extension points are defined as hook specifications (`@hookspec`) and implemented by plugins (`@hookimpl`).
- **SQLAlchemy** with Advanced Alchemy provides ORM access to either SQLite (default) or PostgreSQL for the control-plane database. The control-plane schema stores applications, users, addon associations, and audit records.
- **Dishka** is used for dependency injection across controller, command, and plugin layers.

- **uWSGI** runs the deployed applications in “emperor” mode, where each application is a vassal managed via its `.ini` file. The emperor handles respawn, graceful reloads, and process limits.

The control plane is deliberately a single process. A single-host PaaS has no need to scale the control plane. Operators have one process to monitor, one log file to consult, and one configuration surface. The cost is that the control plane is a single point of failure on the target host; applications continue to run if it dies, but new deployments fail until it restarts.

3.2 Deployment Lifecycle

A deployment is triggered by either `hop3 deploy` (client-driven) or by a `git push` to a bare repository that the server manages on behalf of the application. The pipeline stages, implemented in `hop3/deployers/deployer.py::do_deploy`, are:

1. **Archive upload.** The client ships a source archive to the server; for `git push`, the server receives the tip of the pushed branch.
2. **Configuration parsing.** `Procfile` (if present) and `hop3.toml` (optional) are parsed into an `AppConfig` object (§3.4).
3. **Builder selection.** The `Builder` protocol’s `accept()` method is consulted on each registered builder (LocalBuilder, DockerBuilder, NixBuilder); the first accepting builder wins. Selection can be forced via `[build].builder`.
4. **Toolchain selection.** For LocalBuilder, the `LanguageToolchain` protocol is queried analogously (Python, Node, Go, Rust, Ruby, Java, PHP, generic).
5. **Build.** The toolchain or builder runs in an isolated environment under the `hop3` system user, with per-app isolation (virtualenv or Nix store entry).
6. **Deploy.** The Deployer (uWSGI, Docker Compose, or static) takes the build output and launches the process(es). For uWSGI, this writes a vassal `.ini` file under `uwsgi-enabled/`.
7. **Proxy configuration.** The Proxy plugin writes a per-app site configuration reflecting the application’s port and any custom routes, and asks the proxy to reload.
8. **Health check.** The application is probed on its declared health-check path; failures here are reported via a structured `Diagnosis` naming the specific component that refused to respond.

Idempotency. Each stage is designed so that re-running the pipeline on an already-deployed application converges on the same state as a fresh deployment. A partially failed deployment can be retried without cleanup.

Blast-radius isolation. Per-app isolation at the filesystem (per-app directory), process (per-app uWSGI vassal), and credential level (per-app addon users, per-app environment) means that a misbehaving application cannot observe or interfere with its neighbours through ordinary application-level means. The isolation is POSIX-level, not VM-level or container-level; operators needing stronger isolation should deploy via the Docker builder.

3.3 Plugin Decomposition

The plugin architecture is intentionally narrow: a small number of orthogonal protocols cover every extension point. The decomposition was not selected top-down; it emerged from repeated refactoring as the application catalogue grew. Each protocol corresponds to a point in the deployment pipeline where the behaviour demonstrably differs across applications, and where forcing a single implementation into the core would have required per-application `if`-ladders.

Plugin category	Protocol	Representative implementations
Builder	`accept(app)`, `build(ctx)`	LocalBuilder, DockerBuilder, NixBuilder

Plugin category	Protocol	Representative implementations
Language toolchain	`accept(src)`, `setup()`, `build()`	Python, Node, Go, Rust, Ruby, Java, PHP, generic
Deployer	`deploy(ctx)`	uWSGI, Docker Compose, static
Addon	`provision()`, `destroy()`, `env()`	PostgreSQL, MySQL, Redis, S3
Proxy	`configure(app)`, `reload()`	Nginx, Caddy, Traefik
OS implementation	`package_install()`, `service()`	Debian family, Red Hat family
Health check	`is_configured()`, `check()`	Database connectivity, filesystem, disk

The core imports no concrete plugin module; discovery happens at process startup through Pluggy's hook registry. Each plugin category exposes an `accept()`-style negotiation so that selection is driven by the application's own properties (presence of a Dockerfile, a `Cargo.toml`, a `hop3.nix`, etc.) rather than by external configuration. Forced selection via `hop3.toml` is supported as an override only.

3.4 Configuration Model

Hop3 configuration is layered across two files:

- **`Procfile`** (optional, Heroku-compatible): a line-oriented map from worker role to shell command. This is sufficient for a simple application with a single web process.
- **`hop3.toml`** (optional): a structured TOML file covering metadata (`[metadata]`), build configuration (`[build]`), environment (`[env]`), runtime (`[run]`), addons (`[[addons]]`), health checks (`[healthcheck]`), and, for Nix-based builds, a `[nix]` section driving template selection and parameterisation.

Precedence is explicit: `hop3.toml` overrides `Procfile` where both are present. An application that declares only a `Procfile` deploys with defaults (auto-detected toolchain, uWSGI deployer, Nginx proxy, no addons). An application that declares only a `hop3.toml` is fully specified by that file. The two can coexist.

The design rationale for keeping both formats is that the `Procfile` convention is widely understood and already present in many existing repositories; forcing operators to convert to `hop3.toml` would be a migration tax for no gain. `hop3.toml` is the path for anything beyond the single-web-process default.

3.5 Nix Integration

The Nix integration is the central technical contribution of the current project phase (NGI tasks T1 and T2, see Appendix D). It has two entry points:

Hand-crafted path. The operator places a `hop3.nix` file in the application repository. The file returns an attribute set `{ package, env }` where `package` is a Nix derivation producing `$out/bin/<wrapper>` (the startup entry point) and `$out/hop3/runtime.json` (the runtime contract, see below), and `env` is an attribute set of environment variables exposed to the runtime. This path is maximally flexible and is used for applications whose packaging does not fit the recurring template patterns.

Template-generated path. The hand-crafted path was the starting point, but as the catalogue of `hop3.nix` files grew it became clear that most packagings cluster into a small number of recurring patterns: wrap an existing nixpkgs package, fetch a prebuilt binary, unpack a prebuilt archive, install a Node application, run a PHP application against a web root, build a Python virtualenv, run a Java WAR, install a Ruby application with bundler. The template-generated path extracts these eight patterns and exposes them as parameterisable templates (Appendix B), driven by a concise `[nix]` section in `hop3.toml`. The builder materialises the full `hop3.nix` at build time from the template plus the operator's parameters.

Template extraction was validated empirically: for each pattern, at least one hand-crafted application was ported to the corresponding template and the resulting Nix expression was cross-checked against the

original to ensure no loss of behaviour. The set of eight templates is not a closed taxonomy. Applications that do not fit remain on the hand-crafted path, but it covers the dominant cases in the current catalogue.

The **three-tier reproducibility taxonomy** (Tier-1 nixpkgs-source, Tier-2 `__noChroot`, Tier-3 prebuilt-artefact) is an explicit attribute on each template. `nixpkgs-wrapper` is Tier-1 because it wraps a package whose source nixpkgs itself has already built in a pure sandbox. `php-app`, `python-venv`, `node-prebuilt`, `ruby-bundler` are Tier-2 because their underlying package managers require network access that Nix's pure sandbox does not permit. `prebuilt-binary` and `prebuilt-archive` are Tier-3. The classification is surfaced in the user-facing documentation and in the template metadata so that operators choosing between two viable templates for a given application can make an informed trade-off between reproducibility and packaging effort.

> *Superseded.* The claim that these package managers "require network access that Nix's pure sandbox does not permit" was true of the templates as written; the ecosystems themselves were not the limitation. Each was subsequently sealed by vendoring its dependency set into a fixed-output derivation and installing offline, so Tier 2 now denotes a hermetic source build performed by Hop3, and the tiers rank provenance rather than hermeticity. See ADR 008.

The **runtime contract** (`$out/hop3/runtime.json`) is a small JSON file that the Nix derivation must produce:

```
{
  "workers": { "<role>": "<path to wrapper script>" },
  "env":     { "<name>": "<value>", ... },
  "path":    [ "<path to prepend to $PATH>", ... ]
}
```

This is the only surface the Hop3 runtime inspects after the Nix build; the Nix derivation is otherwise opaque to the runtime. The contract decouples two concerns that would otherwise be entangled: the *build* (how the application's artefacts are produced) and the *integration* (how the Hop3 runtime finds and launches them). Either side can evolve (the templates can add new patterns, the runtime can add new worker roles or lifecycle hooks) as long as the contract is respected. This is why hand-crafted and template-generated paths coexist: they produce the same shape of output.

3.6 Security Model

The control plane requires authentication on every RPC call; addon credentials are encrypted at rest using Fernet AEAD [13] with a key derived from the server's `HOP3_SECRET_KEY` via PBKDF2-HMAC-SHA256. Password hashing uses bcrypt with cost factor 12. SSH tunnelling is supported for the CLI to avoid exposing the RPC endpoint publicly. Rate limiting is applied to authentication endpoints. The system assumes a trusted operator and a single-tenant deployment model; it is not designed as a multi-tenant hosting platform where applications from distinct organisations share a host.

4. Implementation

4.1 Packages

Hop3 is implemented as a Python monorepo with five packages:

Package	Role	Key technologies
`hop3-server`	Control plane running on the target	Litestar, SQLAlchemy, Pluggy, Dishka, uWSGI
`hop3-cli`	Client CLI (`hop3`, `hop`)	Python, Click, JSON-RPC

Package	Role	Key technologies
`hop3-installer`	Installation toolkit (`hop3-install`, `hop3-deploy`)	Python stdlib only; no runtime deps
`hop3-tui`	Terminal UI	Textual
`hop3-testing`	Test framework and `hop3-test` CLI	Pytest, Docker SDK

The `hop3-installer` package deliberately restricts itself to the Python standard library: the installer must run on a freshly provisioned machine before any package manager has produced a working environment, and is distributed as a single bundled file (see §4.7).

4.2 Build and Deployment

The build subsystem entry point is `hop3/deployers/deployer.py::do_deploy`. It orchestrates the stages described in §3.2 and is responsible for emitting structured `Diagnosis` objects on failure. The flow through builder and toolchain is two-level: the builder decides *how* to build (local process tree, Docker container, Nix derivation), and, when the builder is LocalBuilder, the toolchain decides *what* to run (Python virtualenv setup, `npm install`, `go build`, etc.). This split lets Docker and Nix paths bypass the language-toolchain layer entirely while keeping the native path organised.

Language toolchains (`hop3/toolchains/*.py`) use well-known markers for detection: `pyproject.toml` / `requirements.txt` for Python, `package.json` for Node, `go.mod` for Go, `Cargo.toml` for Rust, `Gemfile` for Ruby, `pom.xml` / `build.gradle` for Java, `composer.json` for PHP. Detection can be forced via `[build].toolchain` when an application has multiple markers (e.g. a Python project with a `package.json` for asset building).

Deployers (`hop3/plugins/deploy/*.py`) translate build artefacts into running processes. The uWSGI deployer writes a vassal file under `uwsgi-enabled/` and relies on the emperor to respawn on failure. The Docker Compose deployer generates a per-app `compose.yaml`. The static deployer writes Nginx-served files to a fixed document root.

4.3 Nix Templating System

The Nix template registry lives in `hop3/plugins/build/nix/gen/templates/`. Each template is a Python class implementing:

```
class Template(Protocol):
    name: str
    def generate(self, spec: AppSpec) → str: ...
```

`AppSpec` is a dataclass describing application-level parameters (version, exec target, environment, addon wiring) collected from the `[nix]` section in `hop3.toml`. The template's `generate()` method emits a complete `hop3.nix` expression as a string.

Templates are pure functions: the same `AppSpec` always produces the same Nix output. This makes templates easy to test (a unit test asserts the generated text) and easy to reason about during review. The templates do not use a generic templating engine (Jinja, Mustache); they build Nix expressions by composition of helper functions. This is verbose compared to a text template but prevents template-string errors (escaping bugs, quoting bugs) that are particularly painful in a Nix context, and it permits stronger static analysis of template output.

The `hop3 nix eject <app>` command materialises the generated `hop3.nix` into the application repository and switches the app to the hand-crafted path, for cases where the template output needs local editing. Eject is one-way: a re-templated application returns to the template path and the ejected file is discarded.

4.4 Backing Services

The addon subsystem provides a uniform surface across heterogeneous services. Each addon implements three operations: provision (create the resources, emit connection parameters), destroy (remove them), and env (produce the environment variables the application expects). It is responsible for managing its own state.

The four shipped addons are PostgreSQL, MySQL/MariaDB, Redis, and S3-compatible object storage. The relational addons provision a per-app database, a per-app user, and per-app credentials; the Redis addon provisions a per-app logical database number; the S3 addon provisions a per-app bucket and a per-app access-key pair scoped to that bucket.

A recurring concern across the addons is connectivity from both host-process consumers (applications launched under uWSGI) and containerised consumers (applications launched under Docker Compose). The two consumer classes connect to the database over different network paths, and the addon is responsible for ensuring that authentication succeeds for both paths without the operator having to configure this explicitly. This concern is more visible for relational databases, which have explicit host-based authentication, than for Redis, which binds to the loopback interface by default.

The S3 addon is built on a pluggable backend interface (`hop3/plugins/s3/backend.py`). The initial backend is MinIO; a Garage [14] backend is planned. The abstraction is a direct response to concerns about MinIO's licensing trajectory: operators should be able to switch S3 implementations without changing application code, because S3 is an interface rather than a product. The backend selection is configured server-side and is invisible to applications.

4.5 Reverse Proxy

Three proxy plugins ship: Nginx (default), Caddy, and Traefik. Each plugin generates a per-app site configuration, triggers a proxy reload on configuration change, and manages TLS certificates. Caddy and Traefik integrate directly with Let's Encrypt; the Nginx plugin uses `certbot`. All three produce a per-app configuration file that an operator can inspect and, if necessary, customise via hooks documented in the reference.

4.6 Diagnostic Infrastructure

Two mechanisms in combination address the "why did my deployment fail" question:

Structured diagnosis. A `Diagnosis` dataclass with fields `(component, action, reason, hint)` is emitted at every failure point in the deployment pipeline. The rendered form is `[Component] can't [action]: [reason]. [Hint].`: a shape prescriptive enough that reviewers can spot missing hints during code review. Code paths that previously raised `Abort("deploy failed")` with no context now carry a diagnosis through to the CLI, where it renders as a user-visible block instead of an anonymous stack trace.

Runtime log collection before cleanup. The test runner, on failure, calls `collect_runtime_logs()` *inside* the same try-block as the test itself, so the target application's directory, uWSGI worker logs, build log, generated Nginx configuration, generated uWSGI configuration, and Docker container state and logs are captured *before* the session's cleanup step destroys them. The collected context is stashed on the `TestResult` object and written to a per-test log file. An operator triaging a failed end-to-end run no longer needs to SSH into the target and reconstruct failure state by hand.

4.7 Installer

`hop3-install` bundles itself into a single file that the curl-to-python one-liner on the project website consumes:

```
curl -LsSf https://hop3.cloud/install-server.py | sudo python3 -
```

Two install modes are supported: **single-user** (Nix installed under the user's home, no daemon) and **multi-user** (Nix daemon managed by the host's init system). The mode is chosen automatically based on the detected environment and can be overridden. The installer handles Nix setup, Hop3 server deployment, and post-install health checks; if any stage fails, the failure is reported through the same `Diagnosis` mechanism used by the deployment pipeline, so that the operator sees a consistent failure shape across installation and day-to-day operation.

5. Preliminary Evaluation

The evaluation presented here is **qualitative and architectural**. It documents the reach of the packaged-application corpus, the state of the test infrastructure, and the reproducibility-tier distribution across the Nix-packaged variants. Quantitative measurements (control-plane footprint, deployment latency, Nix closure versus Docker image size) are explicitly deferred to the final report (§7).

5.1 Application Coverage

99 real-application deployments are validated end-to-end on a remote VPS, covering 20+ distinct open-source applications in four build-strategy variants (Docker, native uWSGI, hand-crafted Nix, template-generated Nix). The application mix spans CMS and publishing (WordPress, Ghost, BookStack, Wiki.js, HedgeDoc), collaboration (Etherpad, CryptPad, NextCloud), project management (Kanboard, Focalboard, Vikunja), developer tooling (Gitea, Forgejo, Jenkins, Grafana, Mattermost, Stirling-PDF), analytics and surveys (Matomo, LimeSurvey, Easy Appointments), ERP/CRM (Dolibarr, Invoice Ninja), federation (Matrix Synapse, Mastodon, GoToSocial, Isso), password management (Vaultwarden), and utilities (Adminer, Miniflux, Listmonk, SearXNG, Radicale). Appendix C gives the full catalogue with per-variant coverage status.

A small number of applications are **explicitly deferred** with documented blockers: **Monica** (Laravel Mix / webpack 5 incompatibility, resolved upstream only in Monica v5 which moves to Vite); **SonarQube** (bundled Elasticsearch crashes without kernel-level `vm.max_map_count` tuning, outside the PaaS scope); and **Vaultwarden** docker/native variants (upstream ships only Docker images; compiling from source exceeds the Docker builder timeout and requires a Rust toolchain on the target). Each deferral carries a per-app `DEFERRED.md` under `apps/bad/` listing the blocker and the unblocker. Surfacing these publicly is deliberate: silent gaps are a form of unaddressed debt, and an explicit blocker record invites community contribution and third-party verification.

5.2 Test Infrastructure

The test corpus has four layers:

Layer	Count	Scope
Unit	680	Individual functions, classes, parsers, regex helpers
Integration	240	Component interaction (ORM, plugin loading, RPC dispatch)
System	25	Full app with CLI against a Docker-hosted Hop3 instance
End-to-end (real-app)	99	Full deployment workflows across Docker/native/Nix variants

Unit and integration layers run on every commit; system and end-to-end layers run nightly and on release branches. Cumulative test count is 943 unit + integration + installer tests, plus the 99 end-to-end real-app deployments. The end-to-end suite produces per-test diagnostic log files with the material described

in §4.6; a single test run is self-sufficient for post-mortem analysis, eliminating the "SSH into the target and find out" step that is typical of comparable systems.

5.3 Reproducibility Tiers in the Packaged Corpus

Of the 43 Nix-packaged application variants (23 hand-crafted + 20 template-generated), the tier distribution is approximately:

- **Tier-1 (nixpkgs-source):** Go- and Rust-based applications already available in nixpkgs (Miniflux, Gitea, Forgejo, Grafana, Mattermost, Wiki.js, Vikunja, GoToSocial). Fully reproducible within the nixpkgs-revision contract.
- **Tier-2 (`__noChroot`):** PHP, Python-venv, Node-prebuilt, Ruby-bundler templates; these require network access to a package manager that the strict Nix sandbox does not permit.
- **Tier-3 (prebuilt-archive or prebuilt-binary):** Applications where upstream does not ship source-buildable packaging, or where a per-deployment rebuild would be prohibitively heavy.

The tier is surfaced in the documentation and in the template metadata, so operators and auditors can reason about the reproducibility guarantees of a given deployed application and can make an informed trade-off when a given application can be packaged in more than one way.

5.4 Limits of the Current Evaluation

The following measurements are **not yet available** and are targeted for the final report:

- **Control-plane memory and CPU footprint** compared to K3s, Docker Compose, and bare uWSGI under equivalent workloads.
- **Deployment latency** from `git push` to first healthy response, broken out by build strategy (Docker, native, hand-crafted Nix, template-generated Nix).
- **Nix closure size** compared to equivalent Docker image size, including delta-transfer bandwidth savings when upgrading an application.
- **Application cold-start latency** under uWSGI emperor mode.
- **Reproducibility-in-practice:** bit-for-bit hash comparison across two independent rebuilds of each Tier-1 application in the corpus.

These measurements will be collected on a dedicated benchmark harness operating against the 99-variant corpus. The methodology (workload definition, control for hardware/network variability, statistical treatment) will be documented in the final paper.

6. Discussion

6.1 Design Trade-offs

Hop3 deliberately does not support multi-node orchestration, distributed scheduling, or cross-host service discovery. An operator needing those properties should use Kubernetes or Nomad; Hop3 targets the complement of that population. This is a positioning choice rather than a limitation of the underlying design: the plugin architecture would accommodate a multi-node scheduler, but doing so would require rewiring the deployment pipeline around a consensus layer and would lose the single-process-control-plane simplicity that is the system's main advantage. Accepting that cost is not a free choice, and the empirical evidence suggests that the target population (SMB, non-profit, educational, independent-developer) does not require it.

The decision to offer **both** a uWSGI native path and a Nix path reflects the observation that Nix is the right default for reproducibility but uWSGI is the right default for familiarity. An operator running a handful of

Python Flask applications with a PostgreSQL database benefits little from Nix. An operator packaging a heterogeneous application catalogue benefits greatly. Supporting both doubles the test matrix but avoids forcing a choice the system does not need to force.

The choice of Python for the control plane is a deliberate trade-off. Python's runtime is slower than a compiled alternative (Go, Rust, OCaml) and its memory footprint is larger. These are real costs. The offsetting benefits are ecosystem breadth (the Nix, container, and reverse-proxy ecosystems all have mature Python bindings), rapid iteration, and the low barrier to contribution that matters for an open-source project at this stage. A rewrite in a compiled language is feasible if the control-plane footprint proves to be a barrier in production; the plugin protocols would survive such a rewrite.

6.2 Threats to Validity of the Qualitative Claims

Packaged-application coverage as a proxy for general capability. The claim that Hop3 deploys 20+ real open-source applications is a coverage claim. It demonstrates that the plugin architecture is expressive enough to cover a real ecosystem; whether operators can operate these applications in production under real load with the reliability and latency profiles they need is a separate question. Production-validation data is out of scope for this interim report.

Evaluation corpus selection bias. Applications in the corpus were selected for strategic fit with the project's sovereignty positioning and for availability of plausible self-hosted alternatives to commercial SaaS. The corpus is therefore representative of applications that matter to the target user communities (SMB, non-profit, educational, municipal) but not necessarily of the application ecosystem at large. A broader corpus would strengthen the generalisation claim.

Test-suite completeness as a proxy for correctness. The 99-variant end-to-end suite exercises a deployment flow for each application; it does not exercise sustained operation, upgrade paths under pre-existing data, cross-application interference under resource pressure, or failure-recovery behaviour. The test suite is necessary but not sufficient for operational confidence.

6.3 Limitations Specific to the Nix Integration

The eight templates cover the packaging patterns observed in the current catalogue. Applications built from unusual source layouts, requiring complex multi-stage builds, or producing multiple deployable artefacts still need hand-written `hop3.nix`. The template system reduces the amount of Nix expertise required for the common case; it does not eliminate the need for Nix expertise.

The `__noChroot` escape in Tier-2 templates permits the build to deviate from strict reproducibility in exchange for the ability to package applications whose upstream build system cannot operate in a pure Nix sandbox. This is a pragmatic compromise; operators who require strict Tier-1 reproducibility can inspect the template metadata and choose accordingly. A stricter alternative, vendoring dependencies via nixpkgs-fetched archives, is possible for some of the Tier-2 ecosystems (Python, Node) but would add substantial packaging work per application and is deferred until the quantitative evaluation demonstrates that it matters.

> *Superseded.* The stricter alternative was taken, and for all six ecosystems rather than the two judged feasible here. A fixed-output derivation vendors the dependency set from a committed lockfile, and the application then builds with the network off; `__noChroot` appears in no template. The per-application cost proved to be a lockfile and one hash, far less than the substantial per-application work anticipated above. See ADR 008.

7. Conclusion and Future Work

This report has described Hop3, a plugin-based PaaS for single-server deployment, at the state of April 2026. The principal contribution of the reported period is the completion of the Nix integration: a hand-crafted path, an eight-template auto-generation system, a three-tier reproducibility taxonomy, and a runtime contract decoupling the Nix derivation from the Hop3 runtime. Alongside this, the S3-compatible object-storage addon, the structured `Diagnosis` pipeline, and the per-test runtime-log collection are further contributions that enabled the end-to-end validation of 99 application-variant deployments across 20+ real open-source applications.

The evaluation reported here is qualitative. **The work remaining for the final report** consists of:

1. **Quantitative benchmarking** against K3s, Docker Compose, and bare uWSGI (control-plane footprint, deployment latency, closure/image size, cold-start latency, reproducibility-in-practice). A dedicated benchmark harness will be developed for this.
2. **Multi-service application support**: the current `[run.workers]` model handles same-process-tree workers, but multi-component applications (Mastodon Rails + Sidekiq + Node, NextCloud web + cron, application stacks with dedicated microservices) need per-component resource limits, per-component health checks, and independent scaling. A schema-level specification (ADR 038) is in draft.
3. **External security audit** of the control plane and the addon subsystem.
4. **Operational hardening**: firewall/WAF integration, web UI, user-facing documentation through screencasts, and a first wave of sustained production deployments.
5. **Final report** summarising the architecture and the quantitative evaluation.

A companion paper is in preparation for submission to a systems venue; it is narrower in scope than this report and focuses on the Nix integration and the reproducibility taxonomy.

8. Acknowledgments

This work is supported by the NGI0 Commons Fund through the European Commission's Next Generation Internet initiative. We thank the NLnet Foundation for its continued support and the Nix and nixpkgs communities whose work underpins the reproducible-build path described in this report.

9. References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes. "Borg, Omega, and Kubernetes." *Communications of the ACM*, 59(5), 2016.

[2] Dokku project. "Dokku: The smallest PaaS implementation you've ever seen." https://dokku.com/ (accessed April 2026).

[3] R. Carmo. "Piku: The tiniest PaaS you've ever seen." https://github.com/piku/piku (accessed April 2026).

[4] A. Wiggins. "The Twelve-Factor App." https://12factor.net/ (accessed April 2026).

[5] CapRover project. "CapRover: Scalable PaaS." https://caprover.com/ (accessed April 2026).

[6] E. Dolstra. "The Purely Functional Software Deployment Model." PhD thesis, Utrecht University, 2006.

[7] E. Dolstra, A. Löh. "NixOS: A Purely Functional Linux Distribution." In *Proc. 13th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2008.

[8] Reproducible Builds project. “Reproducible Builds — an independent, nonprofit effort.” https://reproducible-builds.org/ (accessed April 2026).

[9] Cloud Foundry Foundation. “Cloud Foundry: Open-source multi-cloud application platform.” https://www.cloudfoundry.org/ (accessed April 2026).

[10] S. Kuenzer, V.-A. Bădoiu, H. Lefeuvre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, Ş. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, F. Huici. “Unikraft: Fast, Specialized Unikernels the Easy Way.” In *Proc. 16th European Conference on Computer Systems (EuroSys)*, 2021.

[11] Litestar project. “Litestar: Effortlessly build performant APIs.” https://litestar.dev/ (accessed April 2026).

[12] H. Krekel, Pluggy contributors. “Pluggy: A minimalist production-ready plugin system.” https://pluggy.readthedocs.io/ (accessed April 2026).

[13] Cryptography.io. “Fernet (symmetric encryption).” https://cryptography.io/en/latest/fernet/ (accessed April 2026).

[14] Deuxfleurs Association. “Garage: A geo-distributed data store.” https://garagehq.deuxfleurs.fr/ (accessed April 2026).

[15] Hop3 project. Source code and documentation. https://github.com/abilian/hop3 ; https://hop3.cloud (accessed April 2026).

Appendix A: Glossary

Term	Definition
Addon	A backing service (database, cache, object storage) provisioned and managed by Hop3 on behalf of deployed applications.
Builder	A plugin that orchestrates the build of an application, selecting a language toolchain or a dedicated build path (e.g. Docker, Nix).
Deployer	A plugin that turns build artefacts into running processes (uWSGI vassals, Docker containers, static files).
Toolchain	Language-specific build logic, used by the LocalBuilder when a matching language is detected.
Proxy	A reverse proxy routing HTTP(S) traffic to deployed applications (Nginx, Caddy, or Traefik).
Diagnosis	A structured error record `(component, action, reason, hint)` emitted at every failure point in the deployment pipeline.
Runtime contract	The `$out/hop3/runtime.json` file a Nix derivation must produce to be runnable by Hop3.
Tier-1/2/3	The reproducibility taxonomy for Nix-packaged applications, *as of April 2026*: fully sandboxed (1), `__noChroot` escape (2), prebuilt artefact (3). Superseded; see the note at the head of this report; Tier 2 is now a hermetic source build and the tiers rank provenance.

Appendix B: Nix Template Reference

Template	Purpose	Tier
`nixpkgs-wrapper`	Wraps an existing nixpkgs package (no source fetch) with a Hop3 runtime wrapper	1
`prebuilt-binary`	Fetches a single pre-built binary (single file, optional chmod +x)	3
`prebuilt-archive`	Fetches a pre-built tarball or zip and copies files per a file-mapping spec	3
`node-prebuilt`	Wraps a Node application or a Node runtime around a local source	2
`php-app`	PHP (via nixpkgs) with composer, optional artisan/builtin serve, web-root selection	2

Template	Purpose	Tier
`python-venv`	Python virtualenv with pip install	2
`java-war`	JVM (via nixpkgs) running a Servlet WAR	3
`ruby-bundler`	Ruby with bundler install	2

Each template accepts an `AppSpec` dataclass (see `hop3/plugins/build/nix/gen/spec.py`) and emits a full `hop3.nix` expression. Template selection is driven by `[nix].template` in `hop3.toml`.

Appendix C: Packaged-Application Catalog

Category	Applications	Variants covered
CMS / publishing	WordPress, Ghost, BookStack, Wiki.js, HedgeDoc, XWiki	native, docker, nix, nix-gen (mostly 3-4 each)
Collaboration	Etherpad, CryptPad, NextCloud	native, docker, nix
Project mgmt	Kanboard, Focalboard, Vikunja	4/4 where applicable
Dev tooling	Gitea, Forgejo, Jenkins, Grafana, Mattermost, Stirling-PDF	native, docker, nix, nix-gen
Analytics / surveys	Matomo, LimeSurvey, Easy Appointments	4/4
ERP / CRM	Dolibarr, Invoice Ninja	4/4
Federation	Matrix Synapse, Mastodon, GoToSocial, Isso	varies
Security	Vaultwarden (nix only; upstream ships only Docker images)	1/4
Utilities	Adminer, Miniflux, Listmonk, SearXNG, Radicale	4/4

Deferred with documented blockers: **Monica** (Laravel Mix / webpack 5), **SonarQube** (bundled Elasticsearch / kernel-level `vm.max_map_count`), **Vaultwarden** docker and native variants (upstream ships only Docker images). Each deferral carries a `DEFERRED.md` under `apps/bad/` listing the blocker and the unblocker.

Appendix D: NGI Deliverable Mapping

This appendix maps the project's NGI task codes onto the work described in the body. The body uses narrative section references rather than milestone codes; this mapping is provided for reviewers familiar with the project's deliverable taxonomy.

Task	Description	Status (April 2026)	Relevant sections
T1	Nix build plugins (hand-crafted + template-generated)	Delivered (95 %)	§3.5, §4.3, Appendix B
T2	Nix runtime	Delivered (90 %)	§3.5 (runtime contract), §4.7 (installer)
T3	Security & resilience	In progress (90 %; code complete, external review pending)	§3.6, §4.4
T4	Packaged applications	In progress (85 %; 99 variants across 4 strategies)	§5.1, Appendix C
T5	Dissemination & engagement	In progress (70 %; docs + conferences done, final paper 0.6)	This report, companion paper

Appendix E: Architecture Decision Records

The project records its major design decisions as Architecture Decision Records (ADRs) under `notes/adrs/`. Each ADR captures the context, the options considered, the decision taken, and the consequences. The table below summarises every ADR as of April 2026.

Statuses follow a small vocabulary: **Draft** (written but not yet validated against implementation), **Accepted** (decision taken, implementation in production or substantially so), **Final** (decision taken and stable with no pending changes), **Implemented** (same as Final but emphasising the code is shipped), **Superseded by ADR NNN** (replaced by a newer decision), **Deferred** (explicit future work with no current implementation), and **Active** (in-progress decision evolving alongside the code). A 2026-04-14 review promoted several long-standing Drafts to Accepted or Final where the code has overtaken the paperwork, and moved ADR 007 to Superseded where ADR 008 now covers its scope.

#	Title	Status	Scope
001	Config Files	Accepted	Relationship between `Procfile` and `hop3.toml`; file placement and precedence.
002	`hop3.toml` Format	Accepted	Section layout (`[metadata]`, `[build]`, `[run]`, `[env]`, `[[addons]]`, `[healthcheck]`, `[nix]`); shipped vs. reserved fields.
003	Config Parsing and Validation	Accepted (phased)	Phase 1 (dataclass + `@property`) shipped; Phase 2 (schema validation) deferred.
004	Development Tooling	Final	Standardisation on `uv`, `ruff`, `pytest`, REUSE-compliant license headers.
005	Web Terminal	Deferred (post-0.6)	In-browser shell access to a deployed application as an alternative to SSH.
006	Nix Integration	Accepted	Phased integration: Phase 1 + 3 shipped, Phase 2 subsumed by Phase 3, Phase 4 deferred.
007	Nix Builders (nixpkgs mode)	Superseded by ADR 008	Original “Blueprint builder” replaced by the `nixpkgs-wrapper` template.
008	Template-Based Nix Expression Generation	Final	The eight-template code-generation system and the three-tier reproducibility taxonomy (§3.5, Appendix B).
009	Nix Runtime Integration	Deferred (Phase 4; post-0.6)	NixOS module generation, Nix-managed backing services; basic Nix-runtime already covered via ADR 006 + 035.
010	Security and Resilience (Umbrella)	Accepted (umbrella)	Landing ADR pointing to child ADRs 011–014, 024, 029; operational posture summarised.
011	Data Encryption and Protection	Accepted	Credential encryption at rest (Fernet AEAD + PBKDF2-HMAC-SHA256); rotation and HSM deferred.

#	Title	Status	Scope
012	Multi-Factor Authentication	Deferred (post-0.6)	TOTP as the first target; SMS OTP explicit non-goal. Design spec retained.
013	Supply Chain Security and SBOMs	Accepted (phased)	Nix hermetic builds (Tier-1) + tooling declared; automated SBOM emission and signature attestation deferred.
014	Authentication Bootstrap	Final	First-admin provisioning and initial login workflow.
015	Documentation and Community Engagement	Accepted	Documentation site shipped; forum/webinar programmes demoted to candidate future mechanisms.
016	Backup Strategy	Accepted (phased)	Long-term vision; Phase 1 shipped via ADR 024; scheduled/remote/encrypted/incremental deferred.
017	Distributed, Agent-Based Architecture	Draft	No implementation; design not yet fully worked out. Phase 1 scheduled for 0.6 (detailed design in ADR 029, also Draft); Phases 2–4 long-arc with decentralised-federation strand tracked in the research subprogram.
018	CLI-Server Communication	Accepted	JSON-RPC over HTTP(S) or SSH-tunnelled HTTP; streaming via companion SSE channel (ADR 034).
019	Basic CLI Commands	Accepted	Command families (`auth:`, `app:`, `system:`, `addon:`, `backup:`, `admin:`, `server:`, `nix:`); some originally-specified commands explicitly deferred.
020	Pluggable Architecture for Core Deployment Workflow	Final	Pluggy-based plugin host, hook specifications, plugin discovery (§3.3); extended by ADR 030.
021	Proxy Plugin System	Final	Abstraction over Nginx / Caddy / Traefik reverse-proxy configuration.
022	Build and Deployment Plugin System	Final	Builder protocol and deployer protocol; per-stage extension points.
023	Runtime Stack Replacement	Draft	Granian + Caddy + custom PM proposal to replace uWSGI + nginx + supervisor. Nothing implemented; design retained as a future option.
024	Backup and Restore System	Final	Phase 1 implementation of the ADR 016 strategy; shipped via `backup:*` CLI commands.
025	CLI User Experience Improvements	Final	Confirmations, structured output, streaming via ADR 034; help system in ADR 036.
026	Dashboard UI Test Classification	Final	Test-layer taxonomy for dashboard UI tests (file-system tests in `c_system/`, others in `b_integration/`).

#	Title	Status	Scope
027	Configuration System Refactoring for Testability	Final	Separation of the four configuration modules (`HopConfig`, `Config`, `AppConfig`, `Hop3Config`).
028	Pluggy + Dishka Integration	Final	Plugins contribute services to the Dishka container via `get_di_providers()`; replaces global-registry anti-pattern.
029	Application Reconciliation and Health Check System	Draft	No implementation; design proposal scheduled for 0.6 as Phase 1 of ADR 017.
030	Two-Level Build Architecture	Final	Builder decides *how* (local, Docker, Nix); toolchain decides *what* (Python, Node, Go, …).
031	Project Terminology	Active	Ubiquitous-language glossary; most terms stable, two ambiguities open (Blueprint, addon/service).
032	Deployment Strategies and Artifact Lifecycle	Accepted	Design accepted; blue-green / versioned-artefact lifecycle not yet shipped for non-Nix builders.
033	Docker Integration Strategy	Final	DockerBuilder + DockerComposeDeployer in production use across 30+ apps; value-based `localhost` rewrite.
034	Streaming Deployment Logs	Implemented	Real-time SSE streaming of build/deploy output and app logs.
035	Build Artifacts as Runtime Contract	Accepted	`BuildArtifact` + `RuntimeConfig` shipped via NixBuilder; LocalBuilder retrofit deferred.
036	CLI Ergonomics and Help System Design	Draft	Design proposal; awaiting addon/service terminology resolution in ADR 031.
037	Git-Based Deployment Architecture	Implemented	`git push hop3@server:myapp main` flow shipped via Option A: server-side `git-receive-pack`, `git-upload-pack`, `git-hook` commands routed through SSH forced-command; auto-creates apps on first push; e2e tested.
038	Multi-Service Application Support	Active; design phase	`[[component]]` schema for multi-component applications (Mastodon, NextCloud, AppFlowy-class stacks). Phase 1 targeted at 0.6.
039	Python Deploy Strategies: Clarify and Make Explicit	Active; Phase 1 **shipped** (2026-04-15); Phases 2–3 deferred to 0.6	Phase 1 (minimum viable: `--no-dev`, drop `--upgrade`, error on ambiguity, Poetry detection) landed in `toolchains/python.py`. Phases 2–3 (explicit `[build.python].strategy`, lint rules, uv-first tutorials) scheduled for 0.6.

Selected ADRs particularly relevant to this report: **020** (plugin architecture, §3.3), **022** (build/deploy plugins, §4.2), **028** (Pluggy + Dishka, §3.1), **030** (two-level build, §4.2), **008** (Nix template generation, §3.5 and §4.3), **035** (runtime contract, §3.5), **033** (Docker integration, §4.2), **017** / **029** (agent model, future work in §7), and **038** (multi-service applications, future work in §7).

This report was prepared as part of the NGI0 Commons Fund grant for the "Nix Integration for Hop3" project. It is the interim deliverable; the final report is scheduled for Q2 2026.